Table of contents

Deploy Rocket.Chat

Deploy with Docker & Docker Compose 2

Deploy with Docker & Docker Compose

> As of December 15, 2023, Rocket.Chat has ceased support for connections from cloud services and official mobile/desktop apps to workspaces running legacy versions outside our support window. Users on unsupported legacy servers are advised to upgrade to the latest Rocket.Chat version to ensure continued access to cloud, mobile, and desktop applications. **Each Rocket.Chat version is supported for six months post-release.**

Deploying Rocket.Chat using Docker and Docker Compose is a highly recommended deployment method for running Rocket.Chat due to its simplicity and flexibility.

In this guide, you'll learn how to:

- Deploy Rocket.Chat using Docker Compose
- Update Rocket.Chat to a newer version
- Update MongoDB to a compatible version on Podman
- Back up and restore your MongoDB database
- Configure Rocket.Chat using environment variables on Docker

> If you're using the built in Bitnami MongoDB images, refer to this forum post to upgrade to the official MongoDB 8.2 image required for Rocket.Chat 8.0.

Step 1: Install Docker, Docker Compose and Git

This guide is intended for Linux users. Users on macOS or Windows can install Docker Desktop instead.

1. Ensure you have Docker, Docker Compose (Docker Compose v2 is required) and Git installed and operational on your system. If you don't have them installed, you can conveniently set them up in Linux distributions with the command below:

```bash
curl -L https://get.docker.com | sh # Install Docker
sudo apt install git # Install Git
```

> For other OS or alternative installation methods, refer to the official **Docker documentation** and **Git documentation**.

2. To run Docker commands without using `sudo`, add your current user to the Docker group.
   a. Find your current username with this command:

   ```bash
   whoami
   ```

   b. Add your user to the docker group:

   ```bash
   sudo usermod -aG docker $USER
   ```

   You can replace $USER with the username from the previous step.

   c. Finally, reboot the system to for the changes to take effect:

   ```bash
   sudo reboot
   ```

> If your server has an active firewall, you may need to whitelist specific URLs to allow communication with Rocket.Chat's cloud services (e.g., push notifications, marketplace). Refer to the **Firewall Configuration documentation** for the complete list of required URLs.

Step 2: Fetch the Rocket.Chat compose configuration files

The official rocketchat-compose repository contains all the configuration files you need to successfully deploy Rocket.Chat using Docker Compose.

1. Clone the repository using Git and navigate to the cloned directory:

   Bash | Copy

   ```bash
   git clone --depth 1 https://github.com/RocketChat/rocketchat-compose.git
   cd rocketchat-compose
   ```

   This repository includes:

   a. `compose.yml` : The main configuration file for Docker Compose to deploy Rocket.Chat.
   b. `.env.example` : An example enviroment file for customizing your deployment.
   c. Additional configs: Other configuration files to support your deployment, such as Traefik (for reverse proxy and HTTPS) and monitoring tools.

2. To configure your deployment without editing the `compose.yml`, you'll need a dedicated `.env` file. Copy the example file to start your customization:

   Bash | Copy

   ```bash
   cp .env.example .env
   ```

   This command creates the `.env` file where you can now define your deployment variables, such as the desired Rocket.Chat version and your workspace URL.

Step 3: Configure Rocket.Chat

Before launching your Rocket.Chat workspace, you must configure some key variables in the `.env` file you created. Start by opening the `.env` file:

Bash | Copy

```bash
nano .env
```

Set Rocket.Chat version

Modify the `RELEASE` variable to specify the exact version of Rocket.Chat you wish to deploy. Refer to the Rocket.Chat releases page for a list of available stable versions.

Plaintext Copy

```
RELEASE=7.5.0
```

> For production environments, it’s strongly recommended to specify a fixed version number (e.g., 7.5.0) instead of using `latest`. This prevents unexpected issues from automatic, breaking updates.

Configure access URLs

The next step is to configure the `ROOT_URL` and `DOMAIN`. These variables determine how users will access your Rocket.Chat workspace. Configuration differs based on whether you are running a local test or a public production instance.

Local deployment

If you are only testing Rocket.Chat locally on your machine and do not require external access or HTTPS, you can use the default values for these variables:

Plaintext Copy

```
DOMAIN=localhost
ROOT_URL=http://localhost
```

Your workspace will be accessible at `http://localhost:3000` once succesfully deployed.

Production deployment with HTTPs

For a publicly accessible and secure deployment, configure HTTPS using a domain name. Rocket.Chat's default configuration leverages Traefik, a reverse proxy that can automatically manage and generate free TLS/SSL certificates from Let's Encrypt.

> Before proceeding, verify that your domain's A record (or optional CNAME record) is correctly pointed to the public IP address of the server where you are running Docker.

Update the following variables in your `.env` file:

Field	Description	Example
DOMAIN	Your public domain or subdomain name. Do not include https:// or any trailing slashes.	chat.example.com
ROOT_URL	The complete URL your users will use to access the server, including the secure protocol (https://).	https://chat.example.com
LETSENCRYPT_ENABLED	Set to `true` to enable Traefik to automatically obtain and renew certificates.	true
LETSENCRYPT_EMAIL	Your valid email address for certificate renewal and security notifications from Let's Encrypt.	demo@email.com
TRAEFIK_PROTOCOL	Set to `https` to ensure secure connections are enforced by the proxy.	https

Here is an example of the configuration:

Plaintext Copy

```
DOMAIN=chat.example.com
ROOT_URL=https://chat.example.com

LETSENCRYPT_ENABLED=true
LETSENCRYPT_EMAIL=demo@email.com
TRAEFIK_PROTOCOL=https
```

To use Nginx as a reverse proxy instead of Traefik, see Configure Nginx as a reverse proxy for Rocket.Chat.

Enable monitoring

To effectively monitor your workspace, the rocketchat-compose repository includes Prometheus for collecting metrics, Loki for log collection, and Grafana for visualizing metrics and logs through dashboards. This setup allows you to visualize key metrics, track performance, and gain insights into the health of your Rocket.Chat instance.

You can choose to access Grafana either through a **URL path** (e.g., `https://your-domain.com/grafana`) or a **subdomain** (e.g., `https://grafana.your-domain.com`).

Grafana via a path

This option is ideal for local development or if you prefer to keep all services under a single domain, accessing your Grafana dashboard as a subdirectory (e.g., `https://your-domain.com/grafana`).

To configure this, follow these steps in the `.env` file:

1. You can use the default values for the following variables:
    a. `GRAFANA_DOMAIN`: Leave this variable empty.
    b. `GRAFANA_PATH`: Set this to your desired path, ensuring there is no trailing slash (e.g., `/grafana`).

    Plaintext | Copy

    ```
    GRAFANA_DOMAIN=
    GRAFANA_PATH=/grafana
    ```

2. Set a strong password for the Grafana `admin` user by updating the `GRAFANA_ADMIN_PASSWORD` variable:

    Plaintext | Copy

    ```
    GRAFANA_ADMIN_PASSWORD=your_secure_password
    ```

Grafana via a subdomain

Accessing your Grafana dashboard through a dedicated subdomain is the recommended approach for production environments, as it provides a cleaner and more secure access point.

To set this up, follow these steps:

1. Verify that your subdomain's A record is correctly pointed to the public IP address of your server.
2. Open your `.env` file and configure the following Grafana variables:
   a. `GRAFANA_DOMAIN` : Set this to your desired subdomain for accessing the Grafana dashboard. (e.g., `grafana.your-domain.com`).
   b. `GRAFANA_PATH` : Leave this variable empty.

   Plaintext | Copy

   ```
   GRAFANA_DOMAIN=grafana.your-domain.com
   GRAFANA_PATH=
   ```

3. Set a strong password for the Grafana `admin` user by updating the `GRAFANA_ADMIN_PASSWORD` variable:

   Plaintext | Copy

   ```
   GRAFANA_ADMIN_PASSWORD=your_secure_password
   ```

Step 4: Optional configurations

Here are some additional optional configurations you can add to your `.env` file before launching Rocket.Chat:

1. **External MongoDB connection**: If you wish to use an existing or dedicated MongoDB instance (such as a managed service like MongoDB Atlas or a separately hosted deployment) instead of the bundled local database, configure the `MONGO_URL` in the `.env` file like this:

   Plaintext | Copy

   ```
   MONGO_URL=mongodb://<user>:
   <pass>@host1:27017,host2:27017,host3:27017/<databaseNam
   e>?replicaSet=<replicaSet>&ssl=true&authSource=admin
   ```

> For production use, MongoDB must be deployed separately (non-containerized) and configured as a replica set.

2. **Registration token**: If you've received a registration token from our Sales team or Rocket.Chat Cloud, you can use it to automatically register your workspace upon initial startup. To do this, add the token to the `.env` file like this:

   Plaintext | Copy

   ```
   REG_TOKEN=your_token_here
   ```

   > This step is optional, as you can also register the workspace through the **setup wizard UI** after deployment. Once the server successfully starts and registration is complete, the token is permanently stored in the database's cloud settings and is no longer required in the `.env` file. For security reasons, proceed to remove the token from your `.env` file after the first successful deployment.

Step 5: Launch the Rocket.Chat deployment

With your `.env` file configured and saved, you're ready to start your Rocket.Chat workspace.

1. Run the following command to download the all required Docker images (Rocket.Chat, MongoDB, Traefik, etc.) and start the containers in the background:

   Bash | Copy

   ```bash
   docker compose -f compose.database.yml -f compose.monitoring.yml -f compose.traefik.yml -f compose.yml up -d
   ```

   Here's a brief summary of what each file in the command does:

   a. `compose.yml` : Starts the Rocket.Chat application.
   b. `compose.database.yml` : Manages **MongoDB**, the database that Rocket.Chat relies on. It also includes **NATS**, an internal message broker used for communication between services.

c. `compose.monitoring.yml` : Enables monitoring with **Prometheus** (for collecting metrics) and **Grafana** (for visualizing them in dashboards).
d. `compose.traefik.yml` : Manages the **Traefik** reverse proxy, which handles secure routing and automatic HTTPS certificate generation from Let's Encrypt.

You can easily customize your deployment by including only the services you need. For example, if you did not configure monitoring or Traefik (e.g., you are using a separate Nginx proxy), you can omit those `.yml` files:

Bash | Copy

```bash
docker compose -f compose.database.yml -f compose.yml up -d
```

2. To check that all services have successfully started, use this command to list all running containers:

Bash | Copy

```bash
docker ps
```

If you encounter issues during deployment, refer to the following resources for logging and troubleshooting:

- Logging Rocket.Chat
- Deployment FAQ
- MongoDB best practices

Step 6: Access your Rocket.Chat workspace

1. Once your Rocket.Chat instance is deployed, you can access the workspace through your web browser to begin your final configuration:

- **For local testing**: Go to `http://localhost:3000` .
- **For production**: Navigate to the `ROOT_URL` you configured in your `.env` file (e.g., `https://your-domain.com`).

2. After the workspace loads, follow the on-screen prompts to create your first admin user and complete the initial workspace setup. Your workspace and email will be registered to the Rocket.Chat Cloud portal during this process.

Access monitoring dashboard

You can access the **Grafana** dashboard at the path or subdomain you configured in your `.env` file (e.g., `https://your-domain.com/grafana` or `https://grafana.your-domain.com`).

To log in to your Grafana dashboard, use the following credentials:

- **User**: `admin`
- **Password**: The password you set in the `GRAFANA_ADMIN_PASSWORD` variable in your `.env` file.

> The `GRAFANA_ADMIN_PASSWORD` you set in the `.env` file is only applied once during the first setup of the container. To update your password later, you must change it directly within your **Grafana user preferences**.

Step 7: Update file storage

Rocket.Chat stores file uploads using GridFS by default. While this doesn't require extra setup, it's not ideal for production because it increases database load and reduces scalability performance. Rocket.Chat highly recommends using a dedicated object storage service such as Amazon S3, Google Cloud Storage (GCS), or MinIO. Refer to the File Uploads guide for detailed instructions on configuring your preferred file storage solution.

Next steps

Great! You've successfully created your Rocket.Chat workspace with Docker and logged in. Next, explore the following resources to continue using your new workspace:

- User Guides: Learn the basics of your Rocket.Chat account, the types of rooms, and how to communicate with your workspace users.
- Workspace Administration: Administrators and owners can set and manage various configurations.
- Marketplace: Explore the available apps to enhance your workspace.

Update Rocket.Chat on Docker

> Updating the Rocket.Chat image will not affect your data, as it is stored persistently in the separate MongoDB service. Before you proceed, **consult the general guidelines for updating MongoDB**.

To update your Rocket.Chat version:

1. Modify the `RELEASE` variable in the `.env` file to point to the release tag of new version:

   Bash | Copy

   ```bash
   RELEASE=<desired version>
   ```

   For example, `RELEASE=8.0.1` .
2. Run the following command to pull the new image and restart the Rocket.Chat container:

   Bash | Copy

   ```bash
   docker compose -f  compose.yml up -d
   ```

   > Refer to **the official documentation** for more details on Rocket.Chat Docker images.
3. After a few minutes, check that the new container is running:

   Bash | Copy

   ```bash
   docker ps
   ```

   You should see a new Rocket.Chat container running with the updated version tag.
4. Return to your workspace Subscription to verify the updated Rocket.Chat version.

Update MongoDB on Docker

> If you're using the built in Bitnami MongoDB images, refer to this forum post to upgrade to the official MongoDB 8.2 image required for Rocket.Chat 8.0.

Upgrading or downgrading your MongoDB instance may be required when moving between major Rocket.Chat releases. Your Rocket.Chat workspace stores its data in a dedicated MongoDB service. Updating the Rocket.Chat container does not modify your stored data, as long as the MongoDB volume remains intact.

This section explains how to safely change your MongoDB version for Rocket.Chat in Docker, whether you are upgrading to a newer release or downgrading to an earlier one.

Upgrade MongoDB

> Before you proceed, **consult the general guidelines for updating Rocket.Chat**.

If you are moving to a higher MongoDB version, follow the steps below:

1. Ensure that the MongoDB image for the target version is available on Docker Hub under mongodb-community-server. Look for an image tagged `<version>-ubi8` .
2. Set or modify the `MONGODB_VERSION` variable in the `.env` file to point to your desired MongoDB version.

   Bash | Copy

   ```bash
   MONGODB_VERSION=<version>
   ```

   For example, `MONGODB_VERSION=8.2` .
3. Run the following command to pull the new image and restart the MongoDB container:

   Bash | Copy

   ```bash
   docker compose -f  compose.database.yml up -d
   ```

   > Refer to **the official documentation** for more details on Rocket.Chat Docker images.

4. After a few minutes, check that the new container is running:

   Bash | Copy

   ```bash
   docker ps
   ```

You should see a new MongoDB container running with the updated version tag.

Downgrade MongoDB

MongoDB does not support direct binary downgrades if the data files were created by a newer version. To safely downgrade, you must first lower the Feature Compatibility Version (FCV) before changing the MongoDB image. Failing to set the FCV before switching binaries before upgrading will cause MongoDB to exit with an error and the container will repeatedly restart.

> Before you proceed, ensure that the MongoDB image for the targeted version is available on Docker Hub under **mongodb-community-server**. Look for an image tagged `<version>-ubi8` .

To downgrade your MongoDB version in Rocket.Chat, continue with these steps:

1. Before making any changes, create a full database backup:

   Bash | Copy

   ```bash
   docker exec <mongo-container_name> sh -c 'mongodump --archive' > db.dump
   ```

   Replace `<mongo-container_name>` with the current name of the MongoDB container.
2. Enter the MongoDB container:

   Bash | Copy

   ```bash
   docker exec -it  <mongo-container_name> bash
   ```

3. Access the MongoDB shell:

   Bash | Copy

   ```bash
   mongosh
   ```

4. Once in the shell, switch to the database:

```bash
use rocketchat
```

5. Check the current Feature Compatibility Version:

```bash
db.adminCommand({ getParameter: 1, featureCompatibilityVersion: 1 })
```

The returned version should match your current MongoDB binary (for example, `8.2`).

6. Set the FCV to the version you intend to downgrade to:

```bash
db.adminCommand({
  setFeatureCompatibilityVersion: "<version>",
  confirm: true
})
```

7. Confirm that the FCV now matches the target version:

```bash
db.adminCommand({ getParameter: 1, featureCompatibilityVersion: 1 })
```

8. Exit the mongo shell and shut down the MongoDB container:

```bash
docker compose -f compose.database.yml down
```

9. Set or modify the `MONGODB_VERSION` variable in the `.env` file to point to your desired MongoDB version:

```bash
MONGODB_VERSION=<version>
```

For example, MONGODB_VERSION=8.0 .

10. Then, pull and start the database container:

```bash
docker compose -f  compose.database.yml up -d
```

11. After a few minutes, confirm that the new MongoDB container is running:

```bash
docker ps
```

A new MongoDB container is now running with the updated version tag. You can also double-check the MongoDb version in the container with this command:

```bash
docker exec -it <mongo-container_name> mongosh --eval "db.version()"
```

Configure Nginx as a reverse proxy for Rocket.Chat on Docker

This section details how to replace the default Traefik service with Nginx as a reverse proxy to secure your Rocket.Chat deployment with HTTPS, using a free TLS/SSL certificate from Let's Encrypt. This setup requires running Rocket.Chat locally so Nginx can handle all public-facing secure traffic.

> We recommend using Traefik as it's shipped with our official Docker image.

Prerequisites

Before starting, verify the following:

1. Your domain's A record (or optional CNAME record) is correctly pointed to your server's public IP address.

2. You have completed Step 1 and Step 2 to fetch the official Rocket.Chat Docker Compose configuration files.

Deploy Rocket.Chat locally

The next step is to configure Rocket.Chat to run on localhost so that Nginx can act as the front-facing reverse proxy. Continue with these steps to run Rocket.Chat on `http://localhost:3000`:

1. Verify the following variables in your `.env` file are set to these default values:

   Plaintext | Copy

   ```
   DOMAIN=localhost
   ROOT_URL=http://localhost
   LETSENCRYPT_ENABLED=false
   LETSENCRYPT_EMAIL=demo@email.com
   TRAEFIK_PROTOCOL=http
   GRAFANA_DOMAIN=
   GRAFANA_PATH=/grafana
   ```

2. Modify the `RELEASE` variable in `.env` to specify the exact version of Rocket.Chat you wish to deploy. Refer to the Rocket.Chat releases page for a list of available stable versions.

   Plaintext | Copy

   ```
   RELEASE=7.5.0
   ```

   > For production environments, it's strongly recommended to specify a fixed version number (e.g., 7.5.0) instead of using `latest`. This prevents unexpected issues from automatic, breaking updates.

3. Set a strong password for the Grafana `admin` user by updating the `GRAFANA_ADMIN_PASSWORD` variable in `.env`:

   Plaintext | Copy

   ```
   GRAFANA_ADMIN_PASSWORD=your_secure_password
   ```

4. Start your Rocket.Chat services, explicitly omitting the Traefik configuration file (compose.traefik.yml):

Bash Copy

```bash
docker compose -f compose.database.yml -f compose.monitoring.yml -f compose.yml up -d
```

5. Confirm that all necessary services have started successfully:

Bash Copy

```bash
docker ps
```

Rocket.Chat should now be running locally on `http://localhost:3000`.

Get an SSL certificate from Let's Encrypt

Use Let's Encrypt to get a free & open-source SSL certificate by following these steps:

1. Install `certbot` using the relevant package manager for your version of Linux. For Debian-based distributions such as Debian and Ubuntu use :

Bash Copy

```bash
sudo apt update
sudo apt install certbot
```

For RPM-based distributions such as Redhat and Centos, use:

Bash Copy

```bash
sudo yum install yum-utils
sudo yum install nginx
```

> The installation commands may vary based on your Linux distribution and version.

2. Obtain a certificate from Let's Encrypt by running this command (a second or more domains are optional):

```bash
sudo certbot certonly --standalone --email <emailaddress@email.com> -d <domain.com> -d <subdomain.domain.com>
```

Replace `<emailaddress@email.com>` and `<domain.com>` with your actual email adress and domain names. If successful, your certificate files will be stored in `/etc/letsencrypt/live/<your.domain.com>/` .

Configure Nginx web server with TLS/SSL

Now, link your domain to the Rocket.Chat service running locally on port 3000. Nginx will serve as the secure public entry point, handling HTTPS traffic on port 443 and directing requests to the local Rocket.Chat and Grafana services.

1. Install Nginx web server:

```bash
sudo apt-get install nginx
```

2. Backup the default config file for reference:

```bash
cd /etc/nginx/sites-available
sudo mv default default.reference
```

3. Create a new site configuration for Rocket.Chat:

```bash
sudo nano /etc/nginx/sites-available/default
```

4. Paste the following in the new file:

```plaintext
 server {
     listen 443 ssl;
     server_name <ABC.DOMAIN.COM>; #replace <ABC.DOMAIN.COM> with your domain name
     ssl_certificate
```

```
/etc/letsencrypt/live/<ABC.DOMAIN.COM>/fullchain.pem;
#replace <ABC.DOMAIN.COM> with your domain name
     ssl_certificate_key
/etc/letsencrypt/live/<ABC.DOMAIN.COM>/privkey.pem;
#replace <ABC.DOMAIN.COM> with your domain name
     ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
     ssl_prefer_server_ciphers on;
     ssl_ciphers
'EECDH+AESGCM:EDH+AESGCM:AES256+EECDH:AES256+EDH';
     root /usr/share/nginx/html;
     index index.html index.htm;

     # Make site accessible from http://localhost/
     server_name localhost;
     location / {
         proxy_pass http://localhost:3000/;
         proxy_http_version 1.1;
         proxy_set_header Upgrade $http_upgrade;
         proxy_set_header Connection "upgrade";
         proxy_set_header Host $http_host;
         proxy_set_header X-Real-IP $remote_addr;
         proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
         proxy_set_header X-Forwarded-Proto http;
         proxy_set_header X-Nginx-Proxy true;
         proxy_redirect off;
     }

     # Grafana Reverse Proxy
    location /grafana {
        proxy_pass http://localhost:5050/grafana;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $http_host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https; #
```

```
Crucial for secure path access
        proxy_set_header X-Nginx-Proxy true;
        proxy_redirect off;
    }
 }
 server {
     listen 80;
     server_name <ABC.DOMAIN.COM>; #replace
<ABC.DOMAIN.COM> with your domain name
     return 301 https://$host$request_uri;
 }
```

> Replace `ABC.DOMAIN.COM` with your domain name.

5. Test the Nginx configuration to make sure there are no syntax errors:

Bash | Copy

```bash
sudo nginx -t
```

6. If the syntax test is successful, restart Nginx:

Bash | Copy

```bash
sudo systemctl restart nginx
```

> If you have security group restrictions, allow TCP/22 from your current IP for SSH connections and TCP/443 from the IP you plan to use for access.

Access your workspace

1. Once your Rocket.Chat instance is deployed, you can access the workspace workspace securely at `https://<your-domain.com>` .

2. After the workspace loads, follow the on-screen prompts to create your first admin user and complete the initial workspace setup. Your workspace and email will be registered to the Rocket.Chat Cloud portal during this process.

Access monitoring dashboard

You can access the **Grafana** dashboard at the path or subdomain you configured in your `.env` file (e.g., `https://your-domain.com/grafana` or `https://grafana.your-domain.com`).

To log in to your Grafana dashboard, use the following credentials:

- **User**: `admin`
- **Password**: The password you set in the `GRAFANA_ADMIN_PASSWORD` variable in your `.env` file.

> The `GRAFANA_ADMIN_PASSWORD` you set in the `.env` file is only applied once during the first setup of the container. To update your password later, you must change it directly within your **Grafana user preferences**.

Great job! You have successfully enabled TLS on your Rocket.Chat workspace with Nginx.

Set Rocket.Chat deployment environment variable on Docker

Enviroment variables are additional settings that impacts your workspace deployment and configuration. Refer to the official documentation to learn more about Rocket.Chat deployment enviroment variables.

To set up an environment variable in Docker,

1. Open your compose.yml file:

   Bash | Copy

   ```bash
   nano compose.yml
   ```

2. Add the environment variable under the `environment` section of the `rocketchat` service. For example, to override the SMTP Host setting, add:

```yaml
services:
  rocketchat:
    ...
    environment:
      ...
      OVERWRITE_SETTING_SMTP_Host: "my.smtp.server.com"
```

> **Tip**: This is the same section where `ROOT_URL`, `MONGO_URL`, etc is defined.

3. Restart your Rocket.Chat container to apply the changes:

```bash
docker compose stop rocketchat
docker compose rm rocketchat
docker compose up -d rocketchat
```

For a full list of available environment variables, refer to Deployment Environment Variables.

Additional steps

Rocket.Chat Docker images

Rocket.Chat provides an official Docker image repository maintained by the Rocket.Chat team. When setting up your deployment, you specify which Rocket.Chat version to use by adding the image tag.

The base image path is: `registry.rocket.chat/rocketchat/rocket.chat:<tag>`.

Here are the available image tags:

1. **Specific release tags** (recommended for production): You can lock your workspace to a specific Rocket.Chat version using a fixed version tag. This is the best practice for production environments because it prevents unexpected major updates and ensures you only upgrade when you are ready to perform backups and check database compatibility. For example, `registry.rocket.chat/rocketchat/rocket.chat:6.6.0` . Check out the release notes or Docker hub tags for the various Rocket.Chat releases.
2. **Latest tag**: The `latest` tag points to the most recent Rocket.Chat build available in the registry. This may include recent changes and is useful for testing newer versions. Using this tag for production deployments is not recommended.
3. **Develop build**: The `develop` tag tracks the develop branch and includes the newest, untested changes. This tag is intended for development and experimentation only and should not be used in production.

Back up and restore MongoDB data on Docker

Rocket.Chat stores all workspace data in MongoDB. Follow the steps in the commands below to back up and restore your database when running in Docker.

Back up your MongoDB database on Docker

1. List all running Docker containers to find the name of your MongoDB service

   Bash | Copy

   ```bash
   docker ps -a
   ```

   > Take note of your mongo container name (e.g., `rocketchat-mongodb-1`)

2. Run this command to dump the database into a binary file `db.dump` :

   Bash | Copy

   ```bash
   docker exec <container_name> sh -c 'mongodump --archive' > db.dump
   ```

   When successful, you should see `db.dump` file in the current directory.

Restore MongoDB database backup on Docker

To restore the data, use mongorestore inside the MongoDB container, piping the data from your local db.dump file.

Bash Copy

```bash
docker exec -i <container_name> sh -c 'mongorestore --archive' <
db.dump
```

Export to an external database (e.g., MongoDB Atlas)

If you are migrating your data or performing an offline restore, you can use the local `db.dump` file directly with an external MongoDB connection string.

Bash Copy

```bash
mongorestore --uri mongodb+srv://<user>:
<password>@cluster0.w2btl.mongodb.net --archive=db.dump
```

Monitoring and logging your deployment

Effective logging helps you monitor the health and status of your Rocket.Chat deployment. This section covers logging for Rocket.Chat, MongoDB, and reverse proxies like Traefik or Nginx.

To check the status and logs of your containers, follow these steps:

1. Use one of the following commands to view containers in your Docker environment:

   Bash Copy

   ```bash
   docker ps        # Shows currently running containers
   with details like ID, status, and ports
   docker ps -a     # Lists all containers, including
   stopped ones
   docker compose ps  # Lists services managed by Docker
   Compose
   ```

   These commands help you identify the containers running Rocket.Chat, MongoDB, Traefik, and related services.
2. Once you identify the relevant container or service, use one of the following commands to inspect logs:

```bash
docker compose logs -f <service_name>  # Shows realtime logs for a service managed by Docker Compose
docker logs <container_name>       # Displays logs for the specific container
docker logs -f <container_name>    # Shows real-time logs for a container
docker logs --tail 10 <container_name> # Displays the last 10 lines of logs
```

> - Replace `<container_name>` and `<service_name>` with the appropriate values.
> - Rocket.Chat logs to stdout, so there isn't a specific log file within the container. The Docker logging system captures these stdout logs, making them accessible through the docker logs command

3. To inspect MongoDB's health, run:

```bash
docker exec -it <mongodb_container_name> mongosh --eval "db.runCommand({ serverStatus: 1 })"
```

This command returns detailed MongoDB server status information.

> Replace `<mongodb_container_name>` with your MongoDB container name.

4. If using **Nginx** as a reverse proxy, logs are stored inside the container at `/var/log/nginx/`. To view logs, run:

```bash
sudo tail -f /var/log/nginx/access.log  # Access logs
sudo tail -f /var/log/nginx/error.log   # Error logs
```

Encountering issues with your deployment? See the Deployment FAQ for detailed troubleshooting assistance.

Congratulations on successfully deploying Rocket.Chat using Docker! You can now effortlessly communicate with your team members in your workspace. Visit the accessing your workspace guide to configure your workspace and onboard other team members.

Powered by Document360